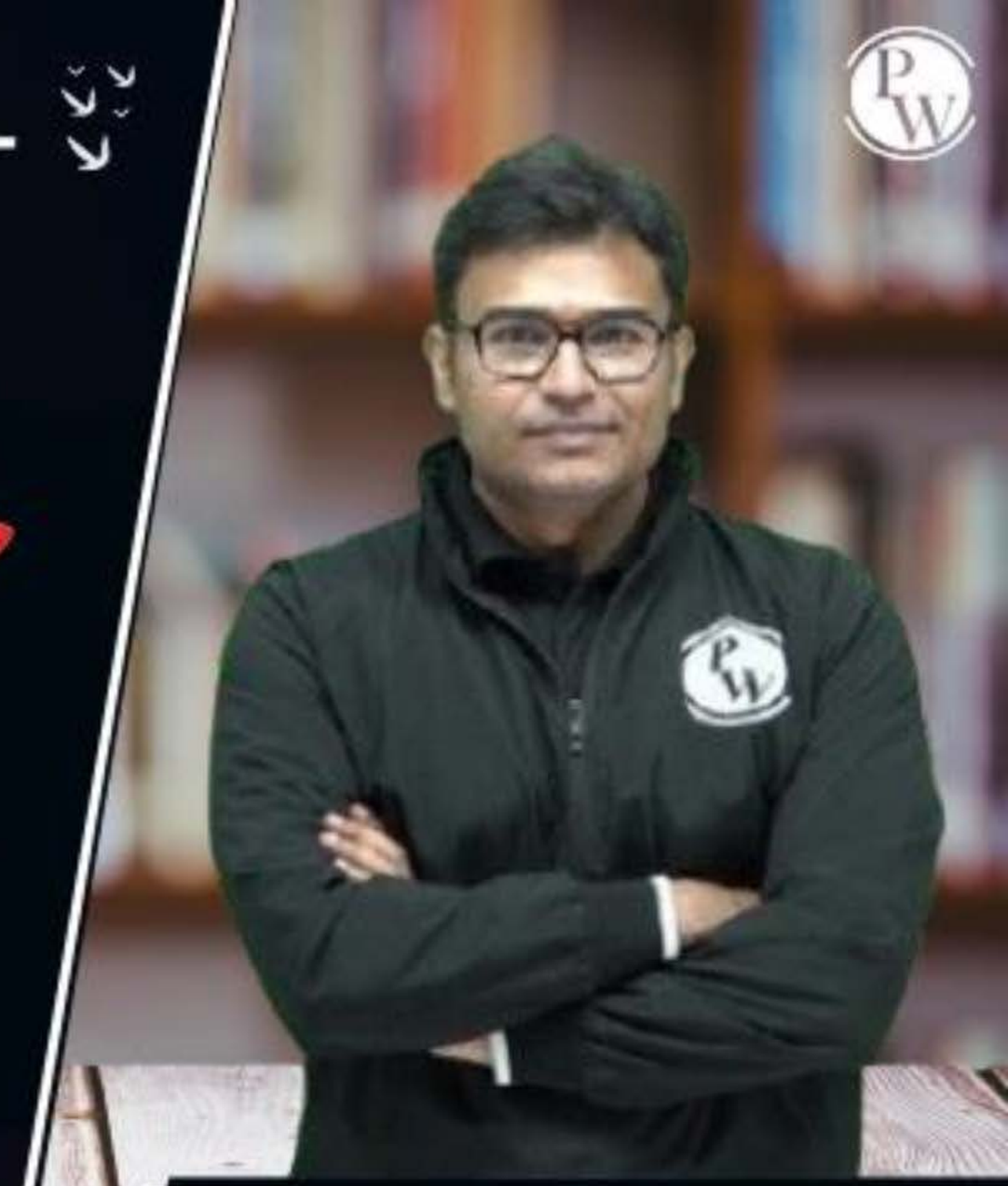
Computer Science & IT

Compiler Design

Parsers

Lecture No. 6



By- Ravindra Sir

Recap of Previous Lecture

- Topic Parsey
- Topic
- Topic

Topics to be Covered

- Topic
- Topic
- Topic

Parsey

Procedure for Converting precedence relation to precedence function table :-

I/p is precedence relation table

Step 1 :- For every terminal 'a' create f_a, g_a

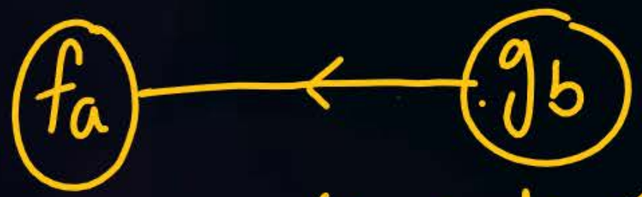
Step 2 :- Divide the Symbols into groups using $(\doteq)$ Symbol

Ex :- If $a \doteq b$ then f_a and g_b are combined into a group $f_a = g_b$

Step 3:- Create a digraph with groups in step '2' as nodes and edges given by $<$ or $>$

For ex :- For example if $a > b$ then add edge as

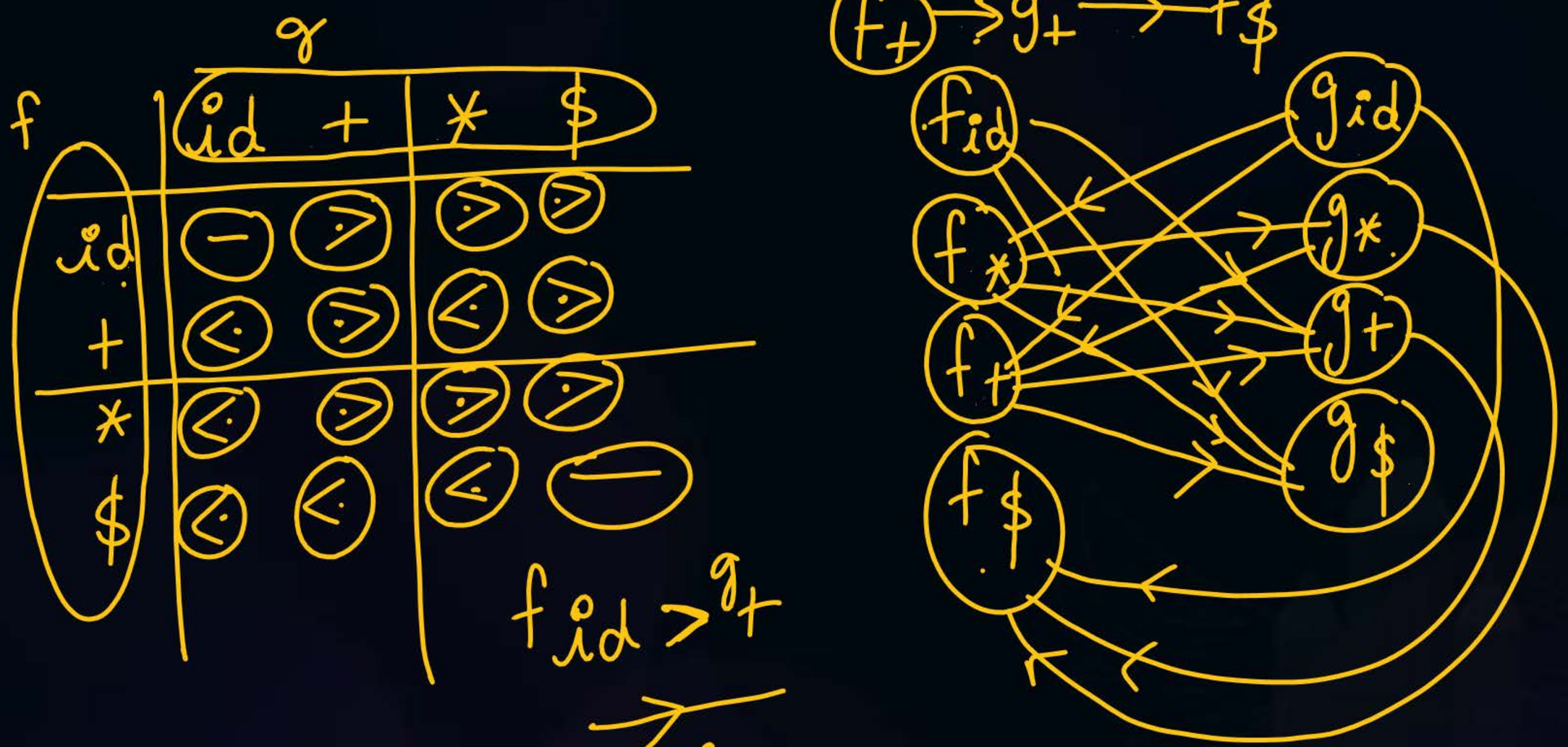
For example if $a < b$ then add edge

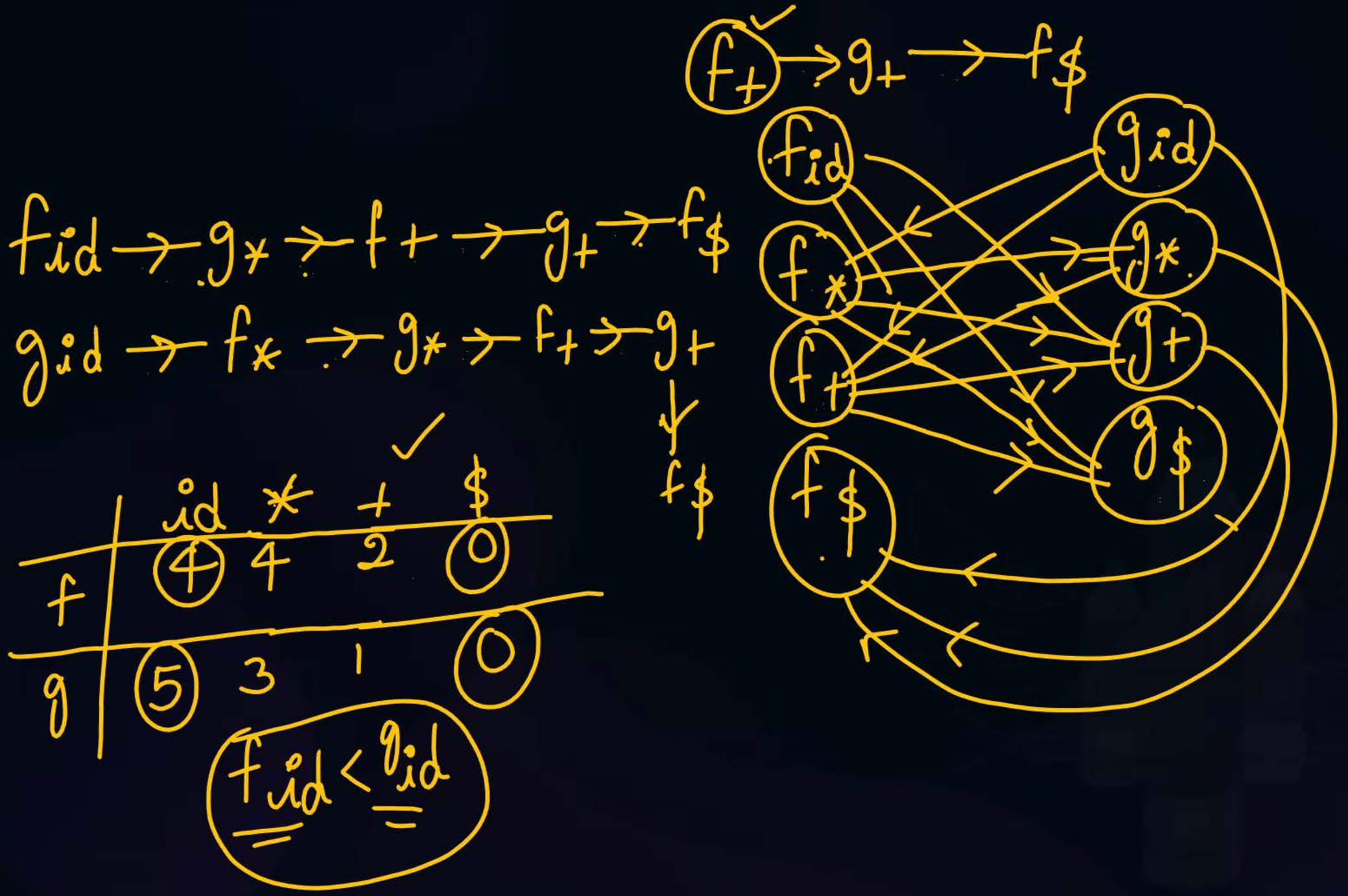


Add an edge for each $<$ or $>$ relation in the table

Step 4: check the digraph for cycles. If there are any cycles, stop the procedure and conclude that the table cannot be converted to function table

Step 5: If there are no cycles, that is if digraph is acyclic, then length of the longest path from fa gives f(a) for each terminal in 'a'


g
f
id
+
*
$
id
+
*
$
$f_{id} > g_+$
$f_+ < g_{id}$
$f_+ \to g_+ \to f_\$$
f_{id}
f_*
f_+
$f_\$$
g_{id}
g_*
g_+
$g_\$$


$f_+ \to g_+ \to f\$$
$f_{id} \to g_* \to f_+ \to g_+ \to f\$$
$g_{id} \to f_* \to g_* \to f_+ \to g_+$
$f\$$
id * + $
f 4 4 2 0
g 5 3 1 0
$f_{id} < g_{id}$
f_{id} g_{id} f_* g_* f_+ g_+ $f\$$ $g\$$

$P \rightarrow SbP / SbS / S$

$S \rightarrow Wb\,S / W$

$W \rightarrow L*W / L$

$L \rightarrow id$

Precedence relation table

	id	*	b	$
id	-	$\doteq>$	$\doteq>$	$\doteq>$
*	$\doteq<$	$\doteq<$	$\doteq>$	$\doteq>$
b	$\doteq<$	$\doteq<$	$\doteq<$	$\doteq>$
$	$\doteq<$	$\doteq<$	$\doteq<$	-

H/W

Precedence function table

$b \doteq< *$

$b \doteq< b$

HW :-

	a	(	)	,	$
a	—	—	>	>	>
(	<	<	=	<	—
)	—	—	>	>	>
,	<	<	>	>	—
$	<	<	—	—	—

$f(=g)$ (f_c, g_c)

digraph

f \ g	a	(	)	,	$
a					
(					
)					
,					
$					

	a	(	)	,	$
f	2	0	2	2	0
g	3	3	0	1	0

Advantage of funtion table :-

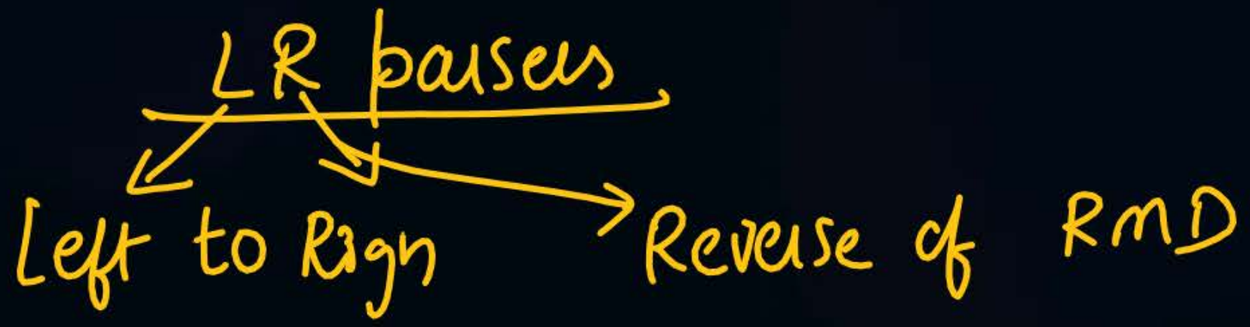
relation table $\rightarrow O(n^2)$

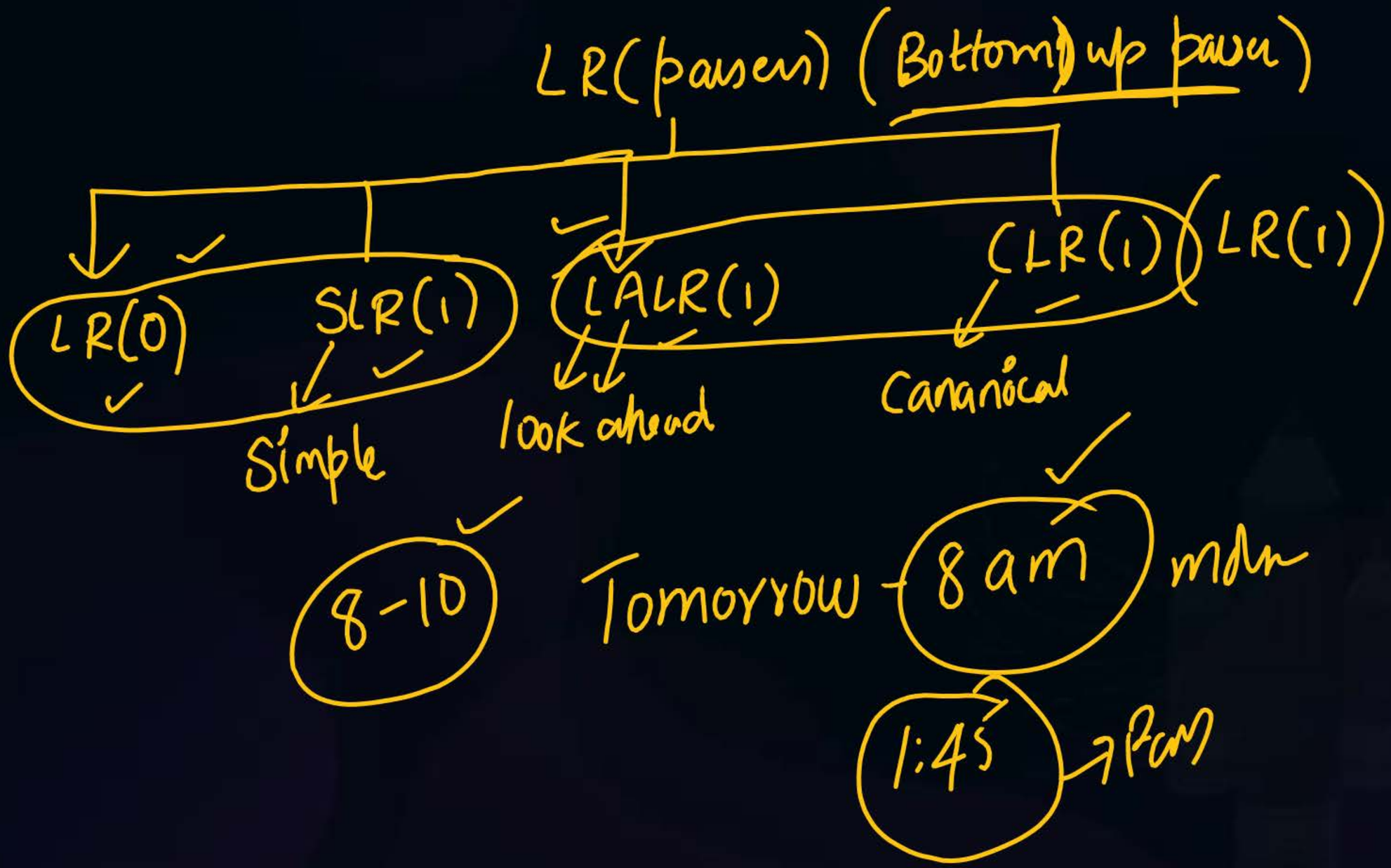
function table $\rightarrow O(n)$

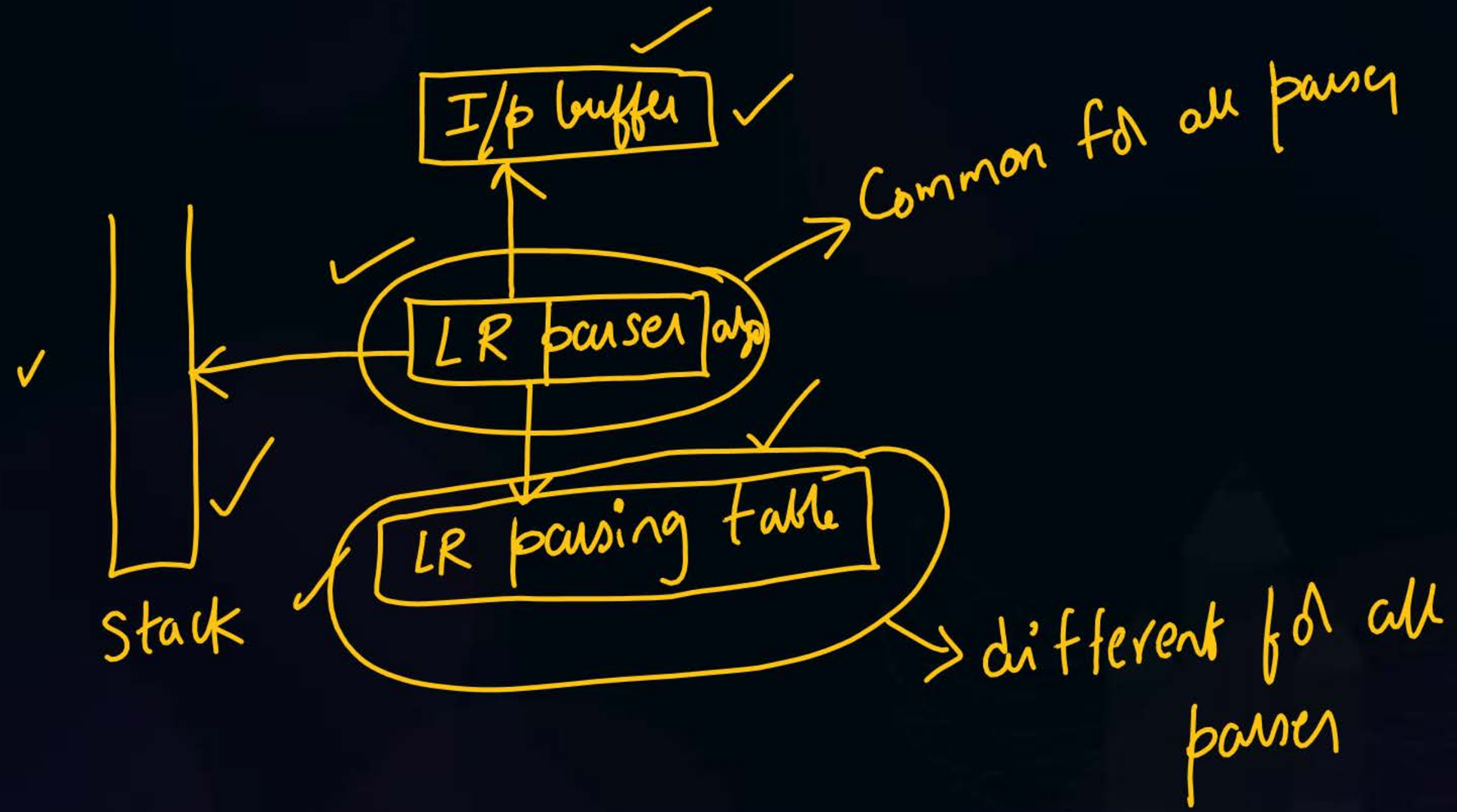
Disadv of function is that it does not have end entries.

So we have to maintain a seperate table

for end entries and see it before seeing function table


LR parsers
Left to Rign
Reverse of RMD


LR(parsers) (Bottom up parsers)
LR(0)
SLR(1)
LALR(1)
CLR(1)
LR(1)
Simple
look ahead
Canonical
8-10
Tomorrow
8 am
mdn
1:45
pm

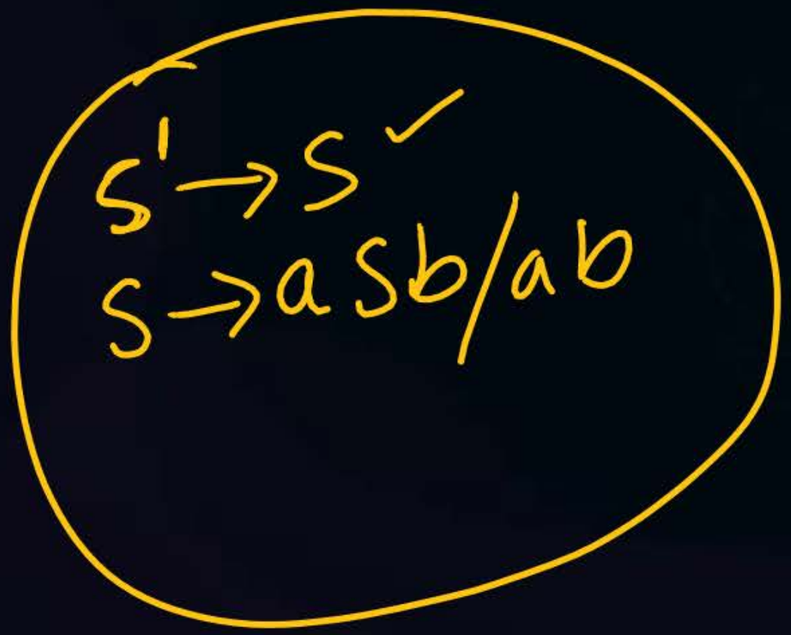

I/p buffer
LR parser algo
LR parsing table
Stack
Common for all parser
different for all parser

To Construct LR(0)/SLR(1) parsing table :-

1) For the given grammar, take augmented grammar
2) Create canonical collection of LR(0) items
3) Draw DFA with set of items and prepare parsing table

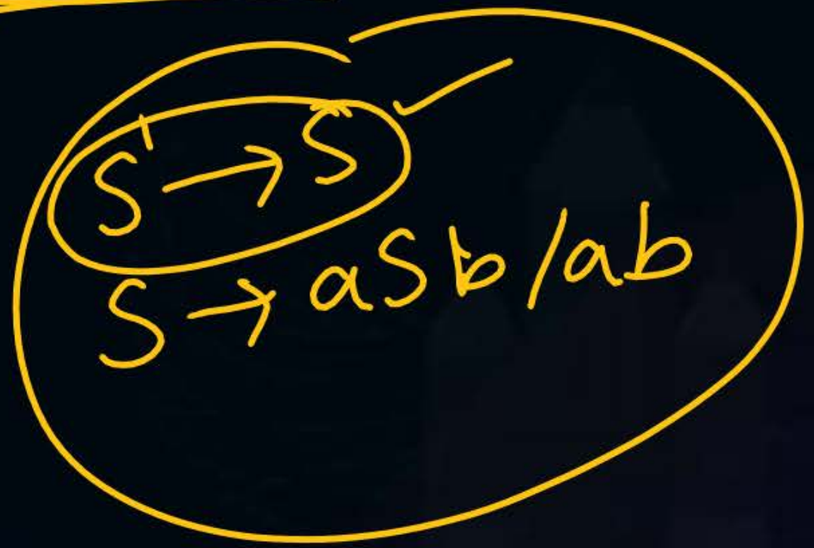
f(=9)

→ Single State



To Construct CLR(1) and LALR(1) parsing table -

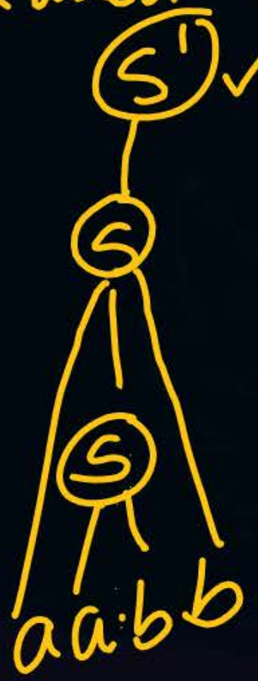
1) fol the given grammar, take augmented grammar
2) Create canonical collection of LR(1) items
3) Draw DFA and then prepare table.



Augmented grammar :-

Given a grammar 'G' with start Symbol 'S' we change the start Symbol to S'. The grammar with newly augmented production is called augmented grammar

$S' \to S$
$S \to asb/ab$



Adding the augmented production helps the parser to understand when to stop parsing and announce the successful completion of the process

Q) why is augmented production added?

In BUP, we start with i/p string and perform reductions until the string is reduced to start symbol. So reducing by start symbol is the final reduction. Sometimes there is a confusion when the start symbol appears in b/w the reduction process. So augmented production is used to avoid confusion.


Roja movie
Paruvam varago
Premam
evare

LR(0) item :-

A production with "dot" at any point on RHS of a production is called LR(0) item

A bottom up parser, starts from i/p string and reads Symbol by Symbol left to right, until a handle is detected. Handle is nothing but RHS of a production. once the handle is found, it reduces. So "dot" tells us how far the RHS is seen by the parser

$X \rightarrow .abc$ → not yet read anything

$X \rightarrow a.bc$ → Read a

$X \rightarrow ab.c$ → Read ab

$X \rightarrow abc.$ → Handle is found so do the reduction.

Any production with '.' at the end is called complete item / final item

8 am - 11am

8am - 12 PM

8am - 11am

2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Parsers

THANK - YOU